

APPLICATION START AND DATABASE COMMANDS

AI SIEM Platform

This PDF contains the important commands required to start the SIEM application, initialize the database, view database records, run demo attack logs, and stop/restart the services.

1. Open Project Folder

```
cd "C:\Users\bpava\OneDrive\Documents\New project\Wazuh-SIEM-Security-Lab\siem-platform"
```

2. Build Docker Images

Run this after Docker Desktop is open. This builds backend and frontend images.

```
docker compose build
```

3. Start Database and Redis

Start only PostgreSQL and Redis first.

```
docker compose up -d postgres redis
```

4. Run Database Migrations

This creates all required database tables.

```
docker compose run --rm backend npm run migrate
```

5. Add Default Demo Data

This creates default admin user, demo source keys, and starter rules.

```
docker compose run --rm backend npm run seed
```

6. Start Full Application

This starts PostgreSQL, Redis, backend, and frontend.

```
docker compose up -d
```

7. Check Running Containers

```
docker compose ps
```

Expected services: postgres, redis, backend, frontend. They should be running or healthy.

8. Open Application

Open this URL in browser:

`http://localhost:5173`

Default login after seed:

Email: `admin@siem.local`
Password: `changeme123`

9. Run Demo Attack Simulation

This sends demo security events to the SIEM collector.

```
.\tools\simulate-attacks.ps1
```

After running it, check Dashboard, Logs, Alerts, and Incidents pages.

10. Backend Health Check

```
curl http://localhost:3001/health
```

PowerShell alternative:

```
Invoke-RestMethod http://localhost:3001/health
```

11. Open PostgreSQL Shell

```
docker compose exec postgres psql -U siem_user -d siem_db
```

12. List Database Tables

Type this inside psql and press Enter:

```
\dt
```

13. View User Records

```
SELECT id, email, full_name, role, is_active, created_at FROM users;
```

14. View Log Source Records

```
SELECT id, name, type, ip_address, is_active, last_seen FROM log_sources;
```

15. View Latest Log Records

```
SELECT id, source_name, severity, category, message, ip_src, user_name, ts
FROM logs
ORDER BY ts DESC
LIMIT 10;
```

16. View Alert Records

```
SELECT id, title, severity, status, source_name, occurrence, created_at
FROM alerts
ORDER BY created_at DESC
LIMIT 10;
```

17. View Incident Records

```
SELECT id, title, severity, status, assigned_to, created_at
FROM incidents
ORDER BY created_at DESC
LIMIT 10;
```

18. View Rule Records

```
SELECT id, name, rule_type, severity, enabled, created_at  
FROM rules  
ORDER BY created_at DESC;
```

19. Exit PostgreSQL Shell

```
\q
```

20. One-Line Database Commands from PowerShell

Use these commands directly from PowerShell without entering psql.

```
docker compose exec postgres psql -U siem_user -d siem_db -c "\dt"
```

```
docker compose exec postgres psql -U siem_user -d siem_db -c "SELECT id, email, role FROM u"
```

```
docker compose exec postgres psql -U siem_user -d siem_db -c "SELECT id, source_name, sever"
```

```
docker compose exec postgres psql -U siem_user -d siem_db -c "SELECT id, title, severity, s"
```

```
docker compose exec postgres psql -U siem_user -d siem_db -c "SELECT id, title, severity, s"
```

21. Stop Application

```
docker compose down
```

22. Start Again Later

```
docker compose up -d
```

23. Restart One Service

```
docker compose restart backend
```

```
docker compose restart frontend
```

24. View Backend Logs

```
docker compose logs -f backend
```

25. View Frontend Logs

```
docker compose logs -f frontend
```

26. View Database Logs

```
docker compose logs -f postgres
```

27. Important Ports

Frontend:	http://localhost:5173
Backend API:	http://localhost:3001
PostgreSQL:	localhost:5432
Redis:	localhost:6379
Syslog UDP:	localhost:5514

28. Notes

Run database commands one by one inside psql. Do not paste \dt and SELECT statements on the same line. If tables are missing, run migrations and seed again.